

Universidad Don Bosco

Minería de Datos y Descubrimiento de Conocimiento para NYC311

Informe de Fase 2 con trazabilidad del proceso KDD

Repositorio del proyecto: github.com/pgap22/dmd-fase2

Datos académicos	Detalle
Asignatura	Data Warehouse y Minería de Datos DMD104
Técnicas	Árbol de Decisión y K-Means
Periodo analizado	Abril a julio de 2026
Fecha de entrega	Septiembre de 2026

Integrantes

Fernando José Lemus Mejía LM212528
Gerardo Rafael Bonilla Saz BS242634
Edson Enrique Chávez Castillo CC242635
César Giovanni López Ortiz LO232756
Juan José Galdámez Soto GS242646

Contenido

El informe documenta la trazabilidad desde NYC Open Data hasta el conocimiento obtenido. También explica la preparación KDD, los modelos, la evaluación y las decisiones sustentadas.

Sección	Contenido
1	Resumen ejecutivo
2	Problema y objetivos
3	Datos y proceso KDD
4	Árbol de Decisión
5	K-Means
6	Conocimiento y recomendaciones
7	Limitaciones y conclusiones
8	Disponibilidad del código y reproducibilidad
9	Referencias
10	Anexos técnicos

1 Resumen ejecutivo

La Fase 2 analiza 1,293,816 solicitudes de NYC311 almacenadas en el Data Warehouse DW_NYC311. El estudio responde dos preguntas. La primera busca identificar solicitudes con riesgo de pertenecer al 25 por ciento de mayores tiempos de resolución. La segunda busca perfiles operativos que combinen volumen, demora y falta de cierre observado.

Resultado	Valor	Lectura para la defensa
Umbral de demora alta	4,329 min	Corresponde al percentil 75 y equivale a unas 72 horas
Árbol optimizado	Recall 95.15%	Detecta 58,549 de 61,531 demoras reales en prueba
Calidad global del árbol	ROC AUC 0.952	Ordena con claridad casos de mayor y menor riesgo
Clustering	K igual a 3	Separa operación rápida, operación lenta y rezago crítico
Separación de clusters	Silhouette 0.421	La estructura es útil, aunque los grupos se solapan

Tabla 1 Resultados principales

El hallazgo operativo central es que la agencia y el tipo de reclamo dominan la predicción. El cluster de resolución prolongada reúne 351,275 solicitudes y representa el principal foco de mejora por su combinación de volumen y demora. El cluster de rezago crítico es más pequeño, pero requiere revisar la trazabilidad de los cierres antes de interpretar sus tiempos.

2 Problema y objetivos

NYC311 recibe solicitudes no urgentes y las asigna a distintas agencias. La diversidad de problemas, canales y zonas produce tiempos de atención muy diferentes. El Data Warehouse de la Fase 1 permite describir ese histórico. Esta fase añade modelos que ayudan a anticipar demoras y a ordenar los perfiles que requieren investigación operativa.

2 1 Objetivo general

Aplicar el proceso de descubrimiento de conocimiento sobre el Data Warehouse de NYC311 para identificar patrones de demora y perfiles operativos útiles para priorizar análisis y seguimiento.

2 2 Preguntas de minería

- Qué características permiten identificar solicitudes con alta probabilidad de demora
- Qué combinaciones de agencia, tipo de reclamo y borough presentan comportamientos similares

El Árbol de Decisión responde la primera pregunta mediante clasificación supervisada. K-Means responde la segunda mediante agrupación no supervisada. Las dos técnicas ofrecen lecturas complementarias y evitan presentar un único modelo como respuesta universal.

3 Origen y trazabilidad de los datos

La información original proviene de 311 Service Requests from 2020 to Present de NYC Open Data, identificado como erm2-nwe9 [1]. La Fase 1 extrajo esos registros mediante ETL, los procesó en las capas raw.Solicitud y stg.Solicitud, y los cargó en DW_NYC311. La Fase 2 no vuelve a la capa RAW. Parte de la tabla de hechos y sus dimensiones para conservar la integración y las reglas aplicadas durante la construcción del Data Warehouse. La vista resultante se llama mining.vw_dataset_fase2.

Etapa	Evidencia utilizada en el proyecto
Fuente externa	NYC Open Data, dataset 311 Service Requests from 2020 to Present, erm2-nwe9 [1]
Extracción Fase 1	ETL desde NYC Open Data hacia raw.Solicitud
Preparación Fase 1	stg.Solicitud estandariza los datos antes de la carga dimensional
Data Warehouse	DW_NYC311 almacena dw.FactSolicitud y las dimensiones relacionadas
Selección Fase 2	mining.vw_dataset_fase2 reúne una fila por solicitud
Intercambio	SSMS exporta resultados.csv con separador de punto y coma
Análisis	pandas carga 1,293,816 filas y scikit-learn ejecuta los modelos [2]
Resultado	El notebook produce métricas, gráficas y conocimiento operativo

Tabla 2 Trazabilidad desde la fuente hasta el conocimiento

3 1 Procedencia de las variables

La vista analítica une dw.FactSolicitud con las dimensiones mediante sus claves sustitutas. La tabla siguiente permite comprobar de dónde sale cada columna usada en minería. La clase demora_alta no existe en la fuente original. La vista la deriva del tiempo observado.

Variable de minería	Procedencia	Transformación en la vista
codigo_solicitud	dw.FactSolicitud	Selección directa
agency	dw.DimAgencia	JOIN por agencia_key
complaint_type	dw.DimProblema	JOIN por problema_key
borough	dw.DimGeografia	LEFT JOIN por geografia_key
status	dw.DimEstado	JOIN por estado_key
open_data_channel_type	dw.DimCanal	LEFT JOIN por canal_key
anio	dw.DimFecha	Atributo calendario
mes	dw.DimFecha	Atributo calendario
dia_semana	dw.DimFecha	Atributo calendario
minutos_resolucion	dw.FactSolicitud	Selección directa
demora_alta	Elaboración propia	CASE sobre el P75 de minutos_resolucion
latitude	dw.FactSolicitud	Selección directa
longitude	dw.FactSolicitud	Selección directa

Tabla 3 Linaje de las 13 variables

3 2 Construcción de la vista y exportación

El script SQL calcula el percentil con PERCENTILE_CONT sobre las solicitudes que sí tienen tiempo de resolución. Después une el hecho con las dimensiones y crea demora_alta mediante un CASE. El siguiente extracto muestra la lógica que produce el dataset, no una consulta inventada para el informe.

```
WITH resolution_percentile AS (
  SELECT DISTINCT PERCENTILE_CONT(0.75) WITHIN GROUP
    (ORDER BY f.minutos_resolucion) OVER () AS p75
  FROM dw.FactSolicitud AS f
  WHERE f.minutos_resolucion IS NOT NULL
)
SELECT f.codigo_solicitud, a.agency, p.complaint_type,
       g.borough, e.status, c.open_data_channel_type,
       fc.anio, fc.mes, fc.dia_semana, f.minutos_resolucion,
       CASE WHEN f.minutos_resolucion > rp.p75 THEN 1
            WHEN f.minutos_resolucion IS NOT NULL THEN 0
            ELSE NULL END AS demora_alta,
       f.latitude, f.longitude
FROM dw.FactSolicitud AS f
JOIN dw.DimFecha AS fc ON f.fecha_creacion_key = fc.fecha_key
JOIN dw.DimAgencia AS a ON f.agencia_key = a.agencia_key
JOIN dw.DimProblema AS p ON f.problema_key = p.problema_key
LEFT JOIN dw.DimGeografia AS g ON f.geografia_key = g.geografia_key
JOIN dw.DimEstado AS e ON f.estado_key = e.estado_key
LEFT JOIN dw.DimCanal AS c ON f.canal_key = c.canal_key;
```

SSMS exportó el resultado como resultados.csv. pandas lo cargó con sep=';'. La forma observada fue 1,293,816 por 13. La suma de 923,081 casos normales, 307,656 demoras altas y 63,079 casos sin clasificar coincide con el total. El archivo no contiene duplicados completos.

3 3 Evidencia del preprocesamiento KDD

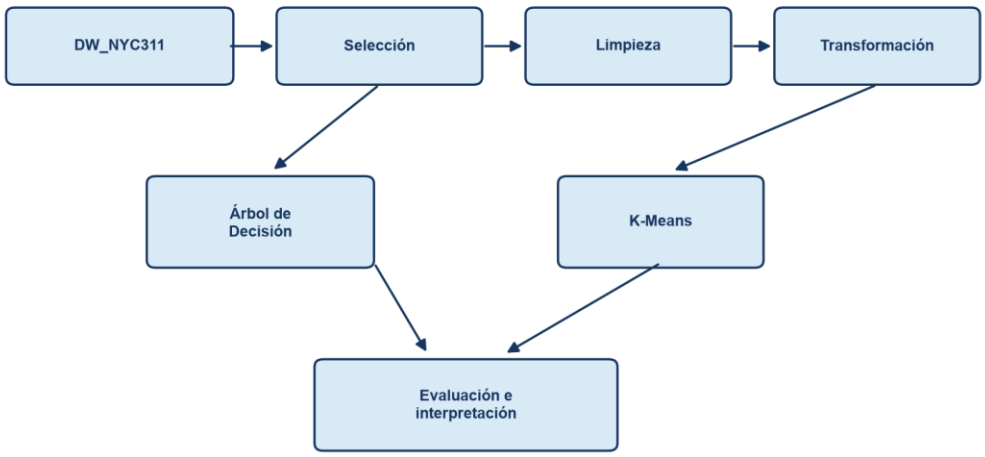


Figura 1 Etapas KDD aplicadas a la Fase 2

Requisito KDD	Aplicación	Justificación
Selección	Vista sobre el hecho y dimensiones	Conserva sólo 13 variables relevantes
Nulos	No se imputan 63,079 tiempos	Las solicitudes todavía no tienen cierre observado
Discretización	P75 igual a 4,329 crea demora_alta	Identifica el 25 por ciento de tiempos más altos
Codificación	OneHotEncoder	Representa categorías sin imponer un orden artificial [2]
Transformación	log1p en volumen y mediana	Reduce la asimetría antes de calcular distancias
Normalización	StandardScaler en K-Means	Lleva las variables a escalas comparables [3]
Control de fuga	Se excluyen minutos, estado e ID	Evita usar el resultado final para predecirlo

Tabla 4 Cumplimiento del preprocesamiento solicitado

3 4 Variable objetivo

Un percentil indica la posición de un valor dentro de una distribución. El percentil 75, o P75, deja aproximadamente 75 por ciento de las observaciones por debajo o igual al corte y 25 por ciento por encima. NYC Open Data no incluye una etiqueta llamada demora_alta. El proyecto necesitaba construirla para entrenar un clasificador y usó el P75 en vez de escoger arbitrariamente un número de horas. El corte se calculó con las 1,230,737 solicitudes que sí poseen tiempo final.

Percentil	Minutos	Lectura aproximada
P25	65	1.1 horas
P50	330	5.5 horas
P75	4,329	72.2 horas o 3 días
P90	21,062	14.6 días
P95	43,399	30.1 días
P99	90,956	63.2 días

Tabla 5 Distribución de minutos de resolución

Una solicitud resuelta recibe demora_alta igual a 1 cuando supera 4,329 minutos. Recibe 0 cuando su tiempo es menor o igual al corte. Las solicitudes sin tiempo final mantienen NULL. El P75 salió de los datos del proyecto. No provino de NYC, del docente ni de scikit-learn.

Clase	Solicitudes	Significado
Normal	923,081	Tiempo observado de hasta 4,329 minutos
Demora alta	307,656	Tiempo observado mayor a 4,329 minutos
Sin clasificar	63,079	No existe tiempo de resolución observado

Tabla 6 Distribución de la variable objetivo

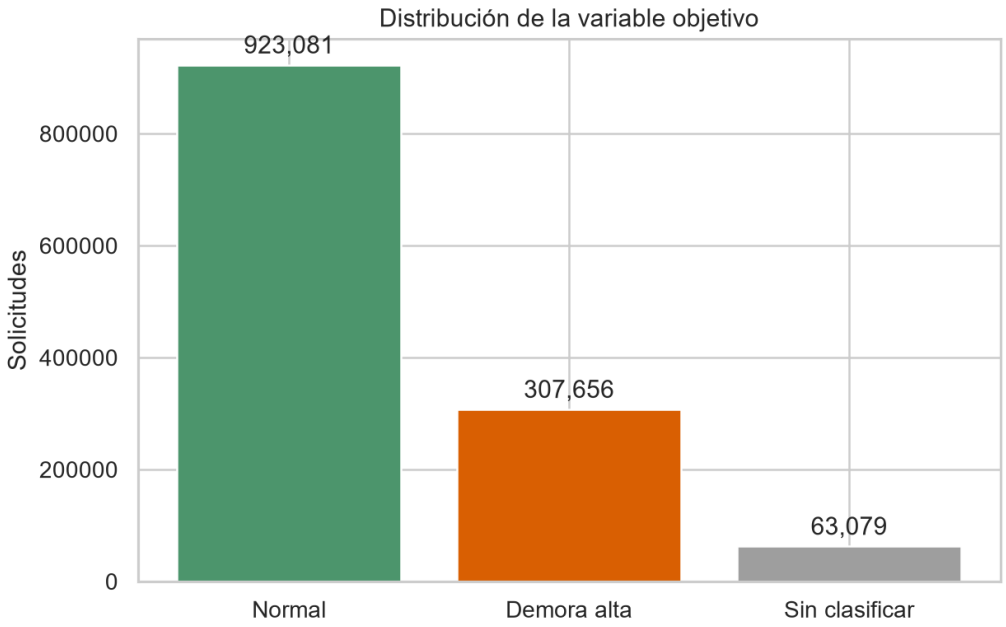


Figura 2 Distribución de la variable objetivo

3 5 Selección de variables

Data leakage o fuga de información ocurre cuando el modelo recibe un dato que en un escenario real todavía no estaría disponible al hacer la predicción. Por esa razón minutos_resolucion no puede ser predictor: demora_alta fue construida usando precisamente ese tiempo. La selección también excluye status porque puede reflejar información posterior al registro.

Variable	Uso	Justificación
agency	Incluida	Representa la operación responsable
complaint type	Incluida	Distingue la naturaleza del problema
borough y canal	Incluidas	Aportan contexto geográfico y de entrada
mes y día de semana	Incluidas	Capturan variación temporal disponible al registrar
minutos resolución	Excluida	Define el objetivo y causaría fuga de información
status	Excluida	Refleja información posterior y está casi constante en casos cerrados
año e identificador	Excluidas	El año no varía y el código no tiene significado predictivo
latitud y longitud	Excluidas	Se reservan para un análisis espacial posterior

Tabla 7 Variables del modelo de clasificación

4 Árbol de Decisión

El conjunto clasificable contiene 1,230,737 solicitudes. Se utilizó 80 por ciento para entrenamiento y 20 por ciento para prueba, con estratificación y semilla 42. Entrenamiento es la parte que usa el modelo para aprender; prueba es la parte apartada para comprobar si generaliza a casos que no vio. Stratify conserva aproximadamente la proporción 75/25 de las clases en ambas particiones.

4 1 Cómo funciona el Árbol de Decisión

El árbol prueba divisiones de las variables para formar nodos cada vez más puros. La pureza se mide con la impureza de Gini. En la expresión, p_i es la proporción de registros de cada clase dentro de un nodo. Un Gini cercano a 0 significa que el nodo contiene casi una sola clase. El árbol elige divisiones que reducen esa mezcla y ayudan a separar demora alta de casos normales.

$$\text{Gini} = 1 - \sum_i p_i^2$$

max_depth limita cuántos niveles puede crecer el árbol. min_samples_leaf evita hojas con muy pocos casos. class_weight igual a balanced aumenta el peso relativo de la clase minoritaria, demora alta, y ayuda a reducir falsos negativos. Las categorías agency, complaint_type y las demás variables de texto se convierten con OneHotEncoder en columnas binarias, sin inventar un orden numérico. El Pipeline encadena esa transformación y el árbol para aplicar siempre los mismos pasos en entrenamiento y predicción [2].

4 2 Ajuste y comparación

GridSearchCV probó todas las combinaciones del espacio siguiente con F1 como criterio [4]. Un hiperparámetro es una configuración definida antes del entrenamiento. La validación cruzada divide los datos de entrenamiento en tres partes: entrena con dos y valida con la restante, rotando la parte validada. No es una validación temporal; se usa como comprobación interna del conjunto de entrenamiento. Tres valores de profundidad, tres mínimos por hoja y dos opciones de pesos producen 18 combinaciones. Con tres particiones se ejecutaron 54 ajustes antes de elegir el modelo final.

Hiperparámetro	Valores probados	Mejor valor
max_depth	6, 10, 15	15
min_samples_leaf	50, 100, 250	50
class_weight	None, balanced	balanced
Criterio	F1 con cv igual a 3	F1 CV 0.787471

Tabla 8 Espacio de búsqueda y selección

El modelo base usó max_depth igual a 6 y min_samples_leaf igual a 100. El modelo elegido aumenta la profundidad y usa pesos balanceados. Ese cambio reduce los falsos negativos, aunque produce más alertas sobre solicitudes que finalmente no pertenecen a demora alta.

Modelo	Exactitud	Precisión	Recall	F1	ROC AUC
Base	86.98%	83.79%	59.41%	69.53%	0.922
Optimizado	86.87%	66.63%	95.15%	78.37%	0.952

Tabla 9 Comparación del Árbol de Decisión

4 3 Lectura de las métricas

Métrica	Qué responde	Resultado optimizado
Recall	Cuántas demoras reales detecta	95.15 por ciento
Precisión	Cuántas alertas terminan siendo demora	66.63 por ciento
F1	Equilibrio entre Recall y Precisión	78.37 por ciento
ROC AUC	Capacidad para ordenar el riesgo	0.952

Tabla 10 Interpretación de métricas

En el conjunto de prueba había 61,531 demoras reales. El modelo identificó 58,549 y dejó 2,982 sin detectar. También produjo 29,328 alertas que finalmente pertenecían a la clase normal. Por tanto, el modelo sirve como filtro de priorización y no como decisión automática.

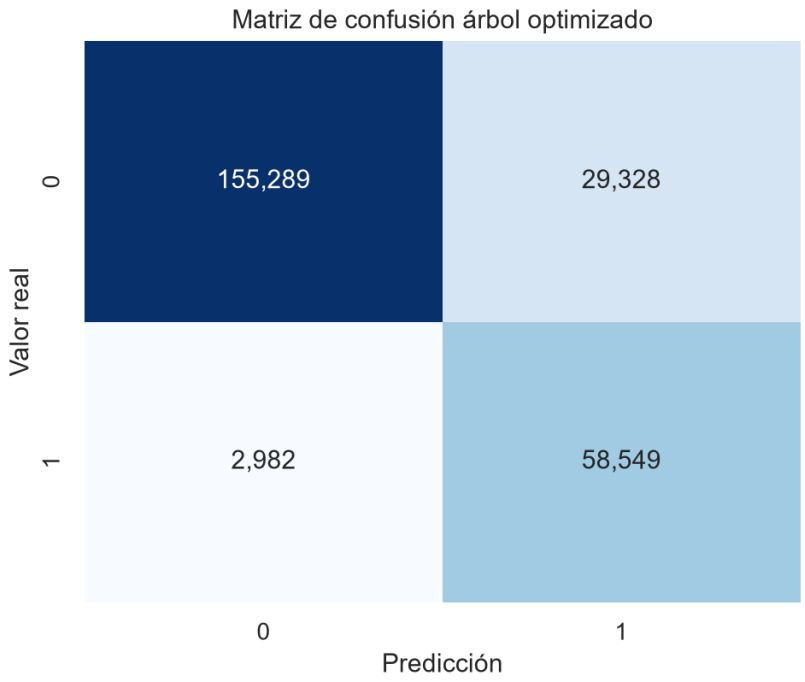


Figura 3 Matriz de confusión del árbol optimizado

4 4 Variables con mayor peso

La agencia concentra 79.95 por ciento de la importancia agregada y el tipo de reclamo 16.27 por ciento. Esta importancia expresa el uso que hace el árbol de cada variable. No demuestra que una agencia cause la demora. La diferencia observada entre operaciones explica por qué un seguimiento segmentado resulta más útil que una única meta para todas las agencias.

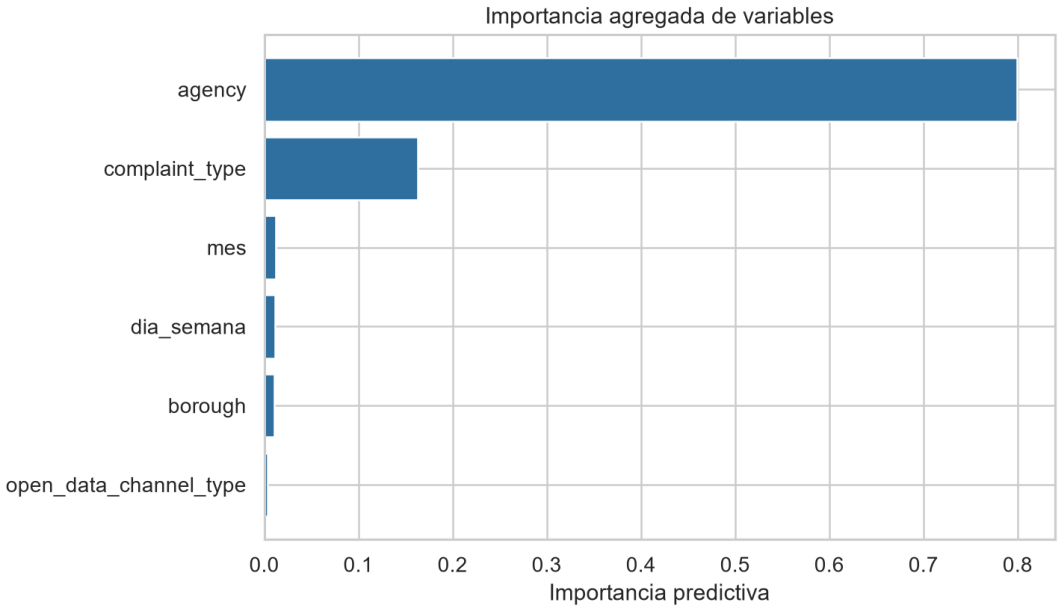


Figura 4 Importancia agregada de variables

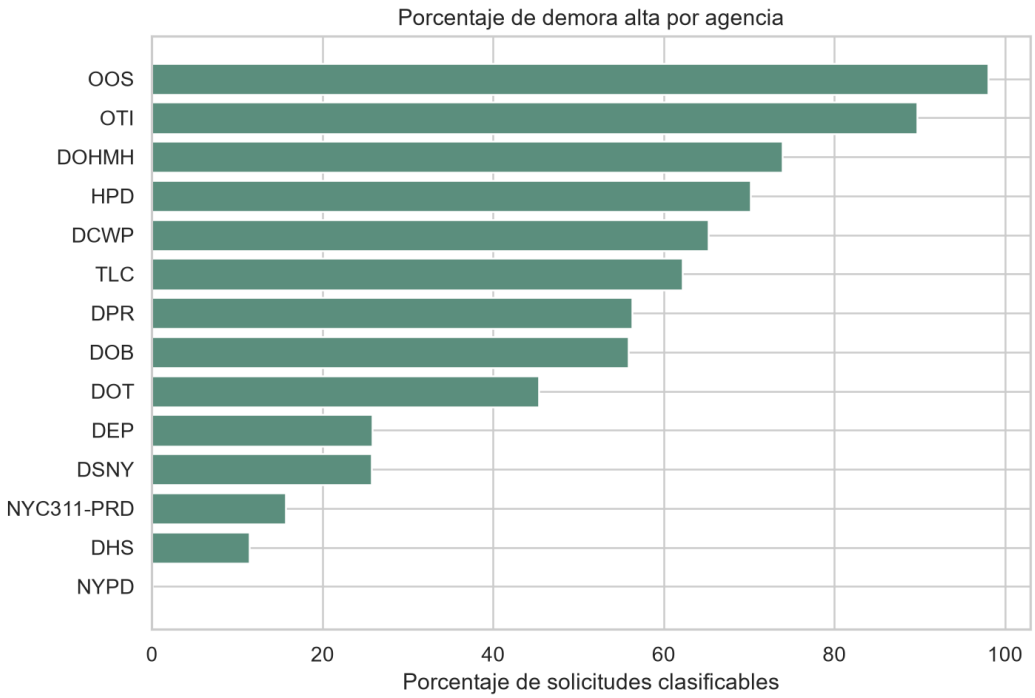


Figura 5 Porcentaje de demora alta por agencia

5 K-Means

K-Means no se aplicó directamente a cada solicitud. Primero se construyeron perfiles por agencia, tipo de reclamo y borough. De 912 perfiles originales, 487 cumplieron el mínimo de 100 solicitudes y 480 conservaron métricas de resolución suficientes para el modelo.

5 1 Cómo funciona K-Means

Cada perfil se representa mediante números. K indica cuántos grupos buscamos; un cluster es uno de esos grupos y μ_j es su centroide, es decir, el punto promedio que resume los perfiles del grupo. K-Means intenta colocar juntos perfiles parecidos y reducir la distancia de cada perfil respecto al centroide de su cluster.

$$J = \sum_{j=1}^{K} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

La función objetivo J es la suma de las distancias cuadradas dentro de los clusters. Esta suma se reporta como Inertia: cuanto menor es, más compactos son los grupos, aunque siempre baja al aumentar K. Silhouette compara la cercanía de un perfil a su propio cluster frente a otros clusters; valores mayores indican mejor separación. Antes de medir distancias se aplica StandardScaler. Sin normalización, una variable numéricamente grande podría dominar las distancias. log1p comprime el volumen y las medianas muy sesgadas sin cambiar su orden.

5 2 Construcción de los perfiles

Cada fila original representa una solicitud. El agrupamiento reúne todas las solicitudes que comparten agencia, tipo de reclamo y borough. Una solicitud se convierte en un perfil junto con las demás filas de la misma combinación. El perfil resume el volumen, la mediana del tiempo, el porcentaje de demora y la proporción sin resolución.

Nivel	Clave	Contenido
Filas originales	HPD + UNSANITARY CONDITION + BRONX	14,286 solicitudes individuales
Perfil agregado	La misma combinación	Mediana 13,285 min y 89.46% con demora
Entrada a K-Means	Un registro del perfil	Volumen, mediana, demora y falta de resolución

Tabla 11 Ejemplo de solicitud a perfil operativo

Variable de perfil	Preparación	Propósito
Número de solicitudes	log1p y estandarización	Representa volumen sin dominar la distancia
Mediana de resolución	log1p y estandarización	Resume el tiempo típico y reduce el efecto de extremos
Porcentaje de demora	Estandarización	Representa la frecuencia de casos por encima del P75

Variable de perfil	Preparación	Propósito
Porcentaje sin resolución	Estandarización	Representa el rezago de cierre observado

Tabla 12 Variables utilizadas por K-Means

5 3 Selección del número de clusters

Se evaluaron valores de K entre 2 y 8. K igual a 3 obtuvo el mayor Silhouette, 0.421321, y produjo tres perfiles con una interpretación operativa clara [3]. El valor indica separación moderada. Los clusters ayudan a explorar prioridades, pero no representan categorías absolutas.

K	Inercia	Silhouette
2	1,135.842	0.384586
3	772.453	0.421321
4	604.326	0.367590
5	455.908	0.365767
6	371.829	0.376295
7	316.815	0.389565
8	284.059	0.380476

Tabla 13 Evaluación de K entre 2 y 8

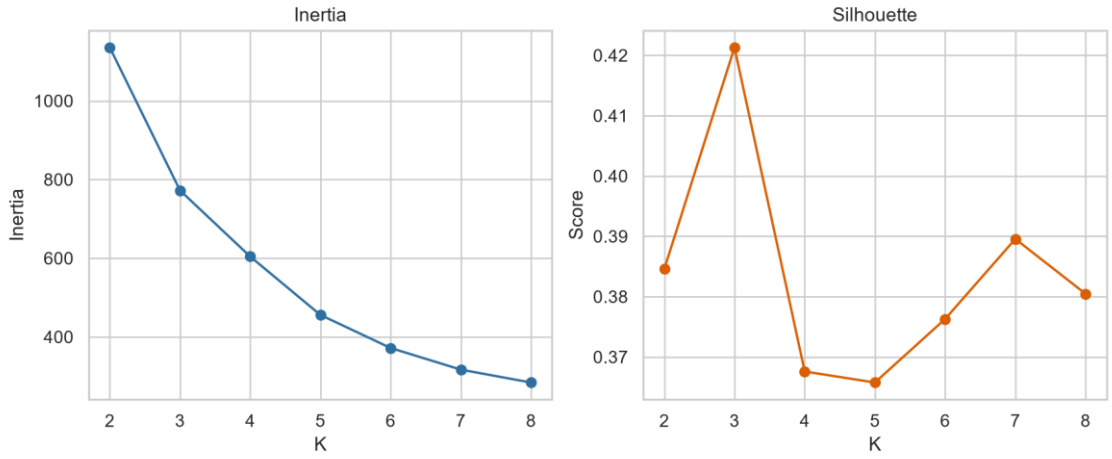


Figura 6 Selección de K mediante inercia y Silhouette

5 4 Perfiles encontrados

Perfil	Perfiles	Solicitudes	Mediana	Demora	Sin resolución
Resolución rápida	206	899,050	143 min	8.94%	1.32%
Resolución prolongada	245	351,275	11,376 min	71.09%	6.09%
Rezago crítico	29	29,467	33,150 min	79.10%	82.72%

Tabla 14 Resumen de los tres clusters

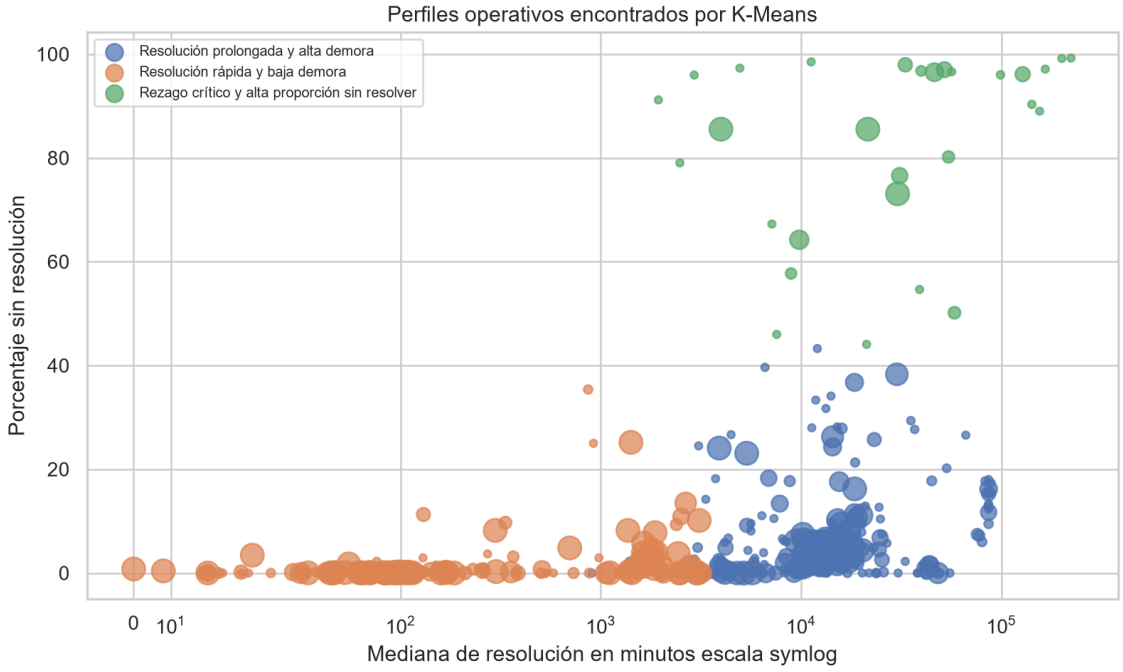


Figura 7 Perfiles operativos encontrados por K-Means

El cluster de resolución rápida concentra la mayoría del volumen y muestra pocos casos demorados. El cluster de resolución prolongada combina volumen relevante con una mediana cercana a 7.9 días. El cluster de rezago crítico reúne menos solicitudes, pero 82.72 por ciento no tiene resolución observada, por lo que primero debe revisarse la trazabilidad de sus estados.

6 Conocimiento y recomendaciones

Evidencia	Interpretación	Uso recomendado
Recall de 95.15%	El árbol deja pocas demoras reales sin alerta	Usarlo como filtro de seguimiento temprano
Agencia y reclamo suman 96.22% de importancia	La operación responsable y el problema separan los riesgos	Definir umbrales y análisis por segmento
351,275 solicitudes en resolución prolongada	Existe volumen suficiente para justificar mejora de proceso	Priorizar los perfiles de mayor volumen dentro del cluster
82.72% sin resolución en rezago crítico	El tiempo final falta en la mayoría de esos perfiles	Auditar estados y cierres antes de fijar metas de tiempo

Tabla 15 Evidencia y uso operativo

6 1 Prioridades de aplicación

- Revisar primero los perfiles de mayor volumen dentro del cluster de resolución prolongada
- Comprobar estados y cierres de los perfiles de rezago crítico antes de comparar tiempos
- Recalcular el percentil 75 y las métricas cuando se incorporen meses nuevos
- Mantener revisión humana sobre las alertas porque una tercera parte puede ser un falso positivo

Estas prioridades convierten cada hallazgo en una comprobación concreta. El análisis no asigna responsabilidad causal ni sustituye la revisión de los procesos de cada agencia.

7 Limitaciones y conclusiones

7 1 Limitaciones

- El periodo cubre abril a julio de 2026 y no permite concluir que el patrón sea anual
- El umbral de 4,329 minutos depende de este periodo y debe recalcularse con nuevos datos
- El P75 fue una regla descriptiva global del periodo antes del experimento; una implementación prospectiva debe calcularlo sólo con el histórico disponible antes del periodo a predecir
- Las solicitudes abiertas no tienen tiempo final y no deben imputarse como si estuvieran resueltas
- La importancia del árbol refleja capacidad predictiva y no una relación causal
- El Silhouette de 0.421 indica solapamiento entre clusters

7 2 Conclusiones

La Fase 2 completa un flujo KDD reproducible sobre el Data Warehouse de NYC311. El Árbol de Decisión optimizado detecta la mayoría de las demoras reales y permite explicar qué variables usa para separar los casos. K-Means identifica un grupo de resolución prolongada con volumen suficiente para priorizar mejoras y otro de rezago crítico que requiere verificar cierres.

La recomendación es utilizar el árbol como alerta para revisión humana, concentrar el análisis de mejora en los perfiles de resolución prolongada y auditar la calidad del estado final en el rezago crítico. Estas acciones respetan los límites del modelo y convierten los resultados en preguntas operativas comprobables.

8 Disponibilidad del código y reproducibilidad

El notebook fuente contiene el código editable y el notebook ejecutado conserva la evidencia de la ejecución validada. La vista SQL define el dataset de minería y resultados.csv es el archivo que se cargó con pandas. Los anexos de este informe son extractos para facilitar la lectura; el notebook completo es la fuente ejecutable del análisis.

Repositorio GitHub: https://github.com/pgap22/dmd-fase2

8 1 Instrucciones básicas

- Ejecutar notebooks/abrir_lab.ps1 desde la raíz del repositorio
- Abrir notebooks/fase2_dmd_nyc311.ipynb en el kernel Python 3
- Comprobar que tmp/data/resultados.csv está disponible o cargarlo en Google Colab
- Ejecutar todas las celdas desde la primera
- Usar RUN_GRID_SEARCH igual a False para una demostración corta con parámetros validados
- Usar RUN_GRID_SEARCH igual a True para repetir la búsqueda completa de 54 ajustes

8 2 Identificación de la evidencia

Elemento	Ubicación o identificador
Vista SQL	tmp/codigo/sql/mining_view_fase2.sql
Dataset exportado	tmp/data/resultados.csv
Notebook fuente	notebooks/fase2_dmd_nyc311.ipynb

Elemento	Ubicación o identificador
Notebook ejecutado	notebooks/fase2_dmd_nyc311_ejecutado.ipynb
SHA-256 notebook fuente	F23311D4AC4892D2A08C7FEFBE1635F6B855CD474BDFAACFCC7F34218A5B3E36
SHA-256 notebook ejecutado	169B4871A97BF61914D70E104F8556D662DD306BF28BBB13017CB2BC29DF8DD2
SHA-256 CSV	E43B66DD36936468E421D91D593F2FE4C16C212F5755DC6B282EA3D818AC4D52

Tabla 16 Archivos que permiten verificar los resultados

Las tablas y gráficas del informe son elaboración propia a partir de resultados.csv y del notebook ejecutado. Las referencias externas sustentan la procedencia del dataset y las definiciones de las herramientas utilizadas.

9 Referencias

[1] NYC Open Data. 311 Service Requests from 2020 to Present. https://data.cityofnewyork.us/d/erm2-nwe9

[2] Scikit-learn. OneHotEncoder. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html

[3] Scikit-learn. Clustering y K-Means. https://scikit-learn.org/stable/modules/clustering.html#k-means

[4] Scikit-learn. GridSearchCV. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[5] Microsoft Learn. SQL Server Integration Services. https://learn.microsoft.com/sql/integration-services/

10 Anexos técnicos

Los anexos reúnen sólo las celdas y fragmentos esenciales para comprobar el flujo. El símbolo [...] indica que se omitieron líneas repetitivas o auxiliares. El código completo permanece en el notebook fuente y en el notebook ejecutado.

Anexo A Construcción del dataset de minería

Propósito. La vista genera una fila por solicitud, calcula el P75 sobre los tiempos observados y deriva la variable demora_alta.

```
WITH resolution_percentile AS (
  SELECT DISTINCT PERCENTILE_CONT(0.75) WITHIN GROUP
    (ORDER BY f.minutos_resolucion) OVER () AS p75
  FROM dw.FactSolicitud AS f
  WHERE f.minutos_resolucion IS NOT NULL
)
SELECT f.codigo_solicitud, a.agency, p.complaint_type,
       g.borough, e.status, c.open_data_channel_type,
       fc.anio, fc.mes, fc.dia_semana, f.minutos_resolucion,
       CASE WHEN f.minutos_resolucion > rp.p75 THEN 1
            WHEN f.minutos_resolucion IS NOT NULL THEN 0
            ELSE NULL END AS demora_alta,
       f.latitude, f.longitude
FROM dw.FactSolicitud AS f
JOIN dw.DimFecha AS fc ON f.fecha_creacion_key = fc.fecha_key
JOIN dw.DimAgencia AS a ON f.agencia_key = a.agencia_key
JOIN dw.DimProblema AS p ON f.problema_key = p.problema_key
LEFT JOIN dw.DimGeografia AS g ON f.geografia_key = g.geografia_key
JOIN dw.DimEstado AS e ON f.estado_key = e.estado_key
LEFT JOIN dw.DimCanal AS c ON f.canal_key = c.canal_key;
```

Lectura. El resultado exportado contiene 1,293,816 solicitudes y 13 variables. Los casos sin tiempo final conservan demora_alta como NULL.

Anexo B Carga y validación del CSV

Propósito. La celda carga el archivo exportado y revisa tamaño, estructura, nulos y duplicados antes de modelar.

```
import pandas as pd
[...]
df = pd.read_csv(DATA_PATH, sep=";")
print(f"Dimensiones: {df.shape}")
display(df.head())
[...]
print("Tipos de datos:")
df.info()
display(df.dtypes.to_frame("dtype"))
print("Valores nulos:")
display(df.isna().sum().to_frame("nulos").query("nulos > 0"))
print(f"Duplicados completos: {df.duplicated().sum():,}")
```

Lectura. df.shape confirma las filas y columnas; df.info y dtypes describen la estructura. Los nulos y duplicados se revisan antes de decidir qué filas pueden participar en cada modelo.

Anexo C Preparación de X e y

Propósito. X contiene las variables que el árbol observa; y contiene la respuesta demora_alta. Sólo se usan filas con y disponible.

```
categorical_features = ["agency", "complaint_type", "borough",
                        "open_data_channel_type"]
numeric_features = ["mes", "dia_semana"]
model_features = categorical_features + numeric_features
classification_df = df.loc[df["demora_alta"].notna()].copy()
X = classification_df[model_features]
y = classification_df["demora_alta"].astype(int)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

Lectura. La separación 80/20 reserva prueba para medir generalización. stratify conserva aproximadamente la proporción de clases en entrenamiento y prueba.

Anexo D OneHotEncoder y Pipeline

Propósito. Las categorías se convierten a columnas binarias y se encadenan con el clasificador.

```
transformer = ColumnTransformer(
    [("categoricas", OneHotEncoder(handle_unknown="ignore"),
      categorical_features)],
    remainder="passthrough",
)
def make_pipeline(class_weight=None, max_depth=None, min_samples_leaf=1):
    return Pipeline([
        ("preprocesamiento", transformer),
        ("modelo", DecisionTreeClassifier(
            class_weight=class_weight, max_depth=max_depth,
            min_samples_leaf=min_samples_leaf, random_state=42,
        )),
    ])
```

Lectura. Sin OneHotEncoder, una categoría como NYPD o HPD no puede pasar al árbol sin asignarle un orden numérico artificial. Pipeline garantiza que la transformación se repita igual al entrenar y al predecir.

Anexo E Árbol baseline

Propósito. El baseline es el punto de referencia antes de ajustar hiperparámetros.

```
baseline = make_pipeline(max_depth=6, min_samples_leaf=100)
baseline.fit(X_train, y_train)
```

Lectura. El baseline limita el árbol a seis niveles y exige al menos 100 casos en cada hoja. Sus métricas se comparan con las del modelo optimizado.

Anexo F GridSearchCV

Propósito. La búsqueda prueba los hiperparámetros definidos en el experimento y selecciona el mejor F1 mediante validación cruzada.

```
grid = GridSearchCV(
    make_pipeline(),
    {
        "modelo__max_depth": [6, 10, 15],
        "modelo__min_samples_leaf": [50, 100, 250],
        "modelo__class_weight": [None, "balanced"],
    },
    scoring="f1", cv=3, n_jobs=-1, verbose=2,
)
grid.fit(X_train, y_train)
optimized = grid.best_estimator_
print("Mejores parámetros:", grid.best_params_)
print(f"Mejor F1 de validación cruzada: {grid.best_score_:.6f}")
```

Lectura. Hay 3 × 3 × 2 = 18 combinaciones. Con cv=3 se realizan 54 ajustes. La mejor configuración fue max_depth=15, min_samples_leaf=50 y class_weight=balanced.

Anexo G Evaluación del Árbol

Propósito. La función calcula predicciones, probabilidades y métricas sobre el conjunto de prueba que el modelo no usó para aprender.

```
def evaluate(model, X_eval, y_eval):
    pred = model.predict(X_eval)
    proba = model.predict_proba(X_eval)[:, 1]
    return {
        "Accuracy": accuracy_score(y_eval, pred),
        "Precision": precision_score(y_eval, pred, zero_division=0),
        "Recall": recall_score(y_eval, pred, zero_division=0),
        "F1": f1_score(y_eval, pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_eval, proba),
        "Matriz": confusion_matrix(y_eval, pred),
        "pred": pred, "proba": proba,
    }
optimized_results = evaluate(optimized, X_test, y_test)
```

Lectura. Accuracy mide acierto total; Precision, cuántas alertas fueron demoras; Recall, cuántas demoras reales se detectaron; F1 equilibra ambas; ROC-AUC evalúa el orden de riesgo. La matriz permite identificar 58,549 verdaderos positivos, 2,982 falsos negativos y 29,328 falsos positivos.

Anexo H Importancia de variables

Propósito. El código recupera las importancias de las columnas transformadas y las vuelve a agrupar por variable de origen.

```
feature_names = optimized.named_steps["preprocesamiento"].get_feature_names_out()
importances = optimized.named_steps["modelo"].feature_importances_
importance_detail = pd.DataFrame({"feature": feature_names, "importance": importances})
importance_detail["feature"] = importance_detail["feature"].str.split("__").str[-1]
importance_detail["source"] = importance_detail["feature"]
for column in categorical_features:
    importance_detail.loc[
        importance_detail["feature"].str.startswith(column + "_"), "source"
    ] = column
importance_by_source = (
    importance_detail.groupby("source", as_index=False)["importance"]
    .sum().sort_values("importance", ascending=False)
)
```

Lectura. La agrupación permite interpretar las variables originales después del OneHotEncoder. agency representa aproximadamente 79.95% y complaint_type aproximadamente 16.27% de la importancia predictiva; estas cifras no prueban causalidad.

Anexo I Construcción de perfiles para K Means

Propósito. K-Means recibe perfiles agregados, no cada ticket individual.

```
profiles = (
    df.groupby(["agency", "complaint_type", "borough"], dropna=False)
    .agg(
        solicitudes=("codigo_solicitud", "count"),
        resueltas=("minutos_resolucion", "count"),
        mediana_resolucion=("minutos_resolucion", "median"),
        promedio_resolucion=("minutos_resolucion", "mean"),
        porcentaje_demora=("demora_alta", "mean"),
    ).reset_index()
)
profiles["porcentaje_sin_resolucion"] = 1 - profiles["resueltas"] / profiles["solicitudes"]
profiles = profiles.loc[profiles["solicitudes"] >= 100].copy()
profiles = profiles.dropna(subset=["mediana_resolucion", "porcentaje_demora"]).copy()
```

Lectura. La agrupación produce 912 perfiles. El filtro de al menos 100 solicitudes deja 487; al eliminar perfiles sin métricas de resolución quedan 480 perfiles para el modelo.

Anexo J Transformación y normalización para K Means

Propósito. Se comprimen las variables muy sesgadas y luego se llevan a una escala comparable.

```
profiles["log_solicitudes"] = np.log1p(profiles["solicitudes"])
profiles["log_mediana_resolucion"] = np.log1p(profiles["mediana_resolucion"])
cluster_features = [
    "log_solicitudes", "log_mediana_resolucion",
    "porcentaje_demora", "porcentaje_sin_resolucion",
]
scaler = StandardScaler()
scaled_profiles = scaler.fit_transform(profiles[cluster_features])
```

Lectura. np.log1p reduce el efecto de extremos de volumen y tiempo. StandardScaler evita que una variable con números grandes domine la distancia euclídea que utiliza K-Means.

Anexo K Evaluación de K

Propósito. El ciclo evalúa K de 2 a 8 con Inertia y Silhouette antes de definir el modelo final.

```
k_metrics = []
for k in range(2, 9):
    candidate = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = candidate.fit_predict(scaled_profiles)
    k_metrics.append({
        "K": k,
        "Inertia": candidate.inertia_,
        "Silhouette": silhouette_score(scaled_profiles, labels),
    })
k_metrics = pd.DataFrame(k_metrics)
```

Lectura. Inertia disminuye al agregar clusters, por lo que se complementa con Silhouette. El mayor Silhouette fue 0.421321 para K=3, una separación moderada y útil para explorar prioridades.

Anexo L K Means final

Propósito. El modelo final asigna un cluster a cada perfil y crea un resumen para interpretar los tres grupos.

```
final_kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
profiles["cluster"] = final_kmeans.fit_predict(scaled_profiles)
raw_cluster_summary = profiles.groupby("cluster").agg(
    perfiles=("cluster", "size"),
    solicitudes=("solicitudes", "sum"),
    mediana_resolucion=("mediana_resolucion", "median"),
    porcentaje_demora=("porcentaje_demora", "mean"),
    porcentaje_sin_resolucion=("porcentaje_sin_resolucion", "mean"),
)
order = raw_cluster_summary.sort_values("mediana_resolucion").index.tolist()
cluster_names = {
    order[0]: "Resolución rápida y baja demora",
    order[1]: "Resolución prolongada y alta demora",
    order[2]: "Rezago crítico y alta proporción sin resolver",
}
profiles["cluster_nombre"] = profiles["cluster"].map(cluster_names)
```

Lectura. K=3 produce los perfiles de resolución rápida, resolución prolongada y rezago crítico. El resumen final usa sus métricas observadas, no etiquetas externas ni una relación causal.